Women Empowerment Platform — Project Explained

Oct 23, 2026 · @omar

1. The idea in one minute

This project is the **backend** (server + database) of a mobile app that connects women who work from home with customers who want homemade food, handmade products or household services.

The problem: skilled women sell through Facebook groups, WhatsApp and word of mouth. That is hard to search, has no order tracking, and customers can't tell who to trust.

Our solution: one platform where providers get verified, list what they sell, receive orders, get paid and get rated, and customers can search, order, pay, review and chat.

Who	What they can do
Guest (not logged in)	Browse categories, providers, food, crafts and services
Customer	Save addresses, place orders, pay, review completed orders, chat with providers
Provider	Upload verification documents; once accepted, sell food, crafts and services, manage slots and orders
Admin	Accept or reject providers, manage categories

What to say first in the evaluation: *"Our backend is a REST API built with Node.js, Express and PostgreSQL. A mobile app sends it HTTP requests and gets JSON back."*

2. Tech stack

We kept the stack small: 4 runtime libraries, no ORM, plain SQL.

Tool	What it does here	Why we chose it
Node.js	Runs our JavaScript on the server	Same language as the web, huge ecosystem

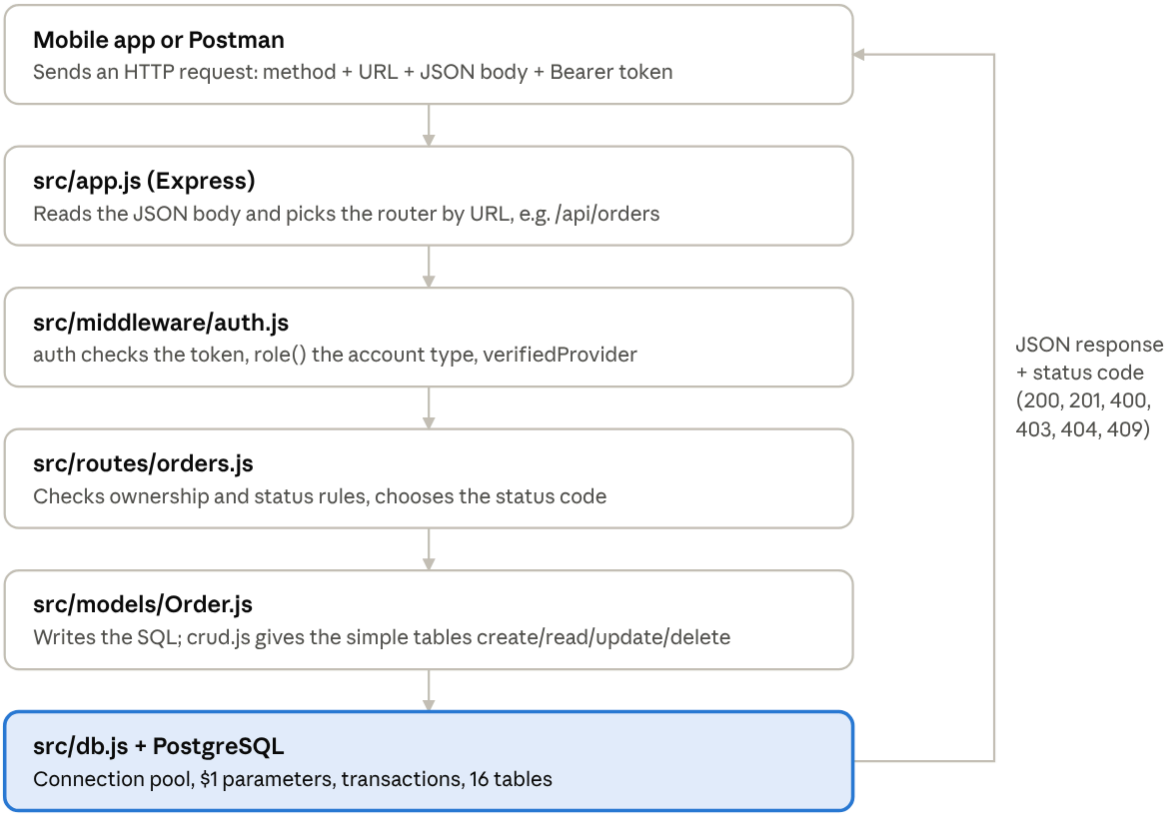
Tool	What it does here	Why we chose it
Express 5	Maps URLs to our functions, reads JSON bodies	Simple, the standard for Node APIs; v5 catches async errors for us
PostgreSQL	Stores all data in 16 related tables	Relational data (orders → items → providers) needs foreign keys and transactions
pg	The driver that sends SQL to PostgreSQL	Lets us write real SQL with safe `$1` parameters
bcryptjs	Hashes passwords	A stolen database doesn't reveal passwords
jsonwebtoken (JWT)	Login token sent with every request	No server sessions to store; works well for mobile apps
PGlite (tests only)	A real Postgres running inside Node	`npm test` needs no installed database
Postman	Sends requests to the API by hand or as a test run	Shows every endpoint working in the evaluation

Why no ORM (Sequelize/Prisma)? Our models write SQL directly. It is less magic, easier to explain line by line, and the database itself enforces the rules.

3. How a request travels through the code

Every request goes through the same layers, each with one job. Example: a customer places an order with `POST /api/orders`.

Every request passes the same layers down to the database and back



request flow · 6 layers

If anything goes wrong on the way down (bad input, a database rule broken), the error jumps to one error handler at the bottom of `src/app.js`, which turns it into the right status code and a clear message.

```
src/
  server.js          creates the tables, starts the server
  app.js             Express app: all routes + the error handler
  db.js              Postgres connection, query helpers, transactions
  schema.sql         the 16 tables (our ERD)
  seed.sql, seed.js  demo data (npm run seed)
  middleware/auth.js login token, role and verified-provider checks
  models/            one file per table; crud.js and item.js are shared
  routes/            one file per resource; items.js serves foods, crafts,
services
test/api.test.js     end-to-end test (npm test)
postman_collection.json
```

4. The database (16 tables from our ERD)

Every entity in the ERD is one table in `src/schema.sql`. The server runs that file on start, so the tables create themselves.

Group	Tables	Key relationships
People	`users`, `customers`, `service_providers`, `user_addresses`	Every account is a `users` row; a customer or provider row links to it 1-to-1 by `user_id`
Trust	`verification_documents`	Many documents per provider; status lives on `service_providers.verification_status`
Catalogue	`categories`, `food_items`, `craft_items`, `domestic_services`, `appointment_slots`	Each item belongs to one provider and one category; categories can have a parent; slots belong to a service
Orders	`orders`, `order_items`, `payment_transactions`	One order = one customer + one provider; many lines; many payment attempts
Feedback	`reviews`	One review per completed order (`UNIQUE order_id`)
Chat	`chat_conversations`, `chat_messages`	One conversation per customer–provider pair; many messages

Things worth pointing at in the evaluation:

- **Primary keys** are `SERIAL` (auto-numbered), named like the ERD: `user_id`, `order_id`…
- **Foreign keys** (`REFERENCES`) stop orphan data: you can't order an item that doesn't exist.
- **CHECK constraints** reject bad values in the database itself: `price >= 0`, `rating BETWEEN 1 AND 5`, `end_time > start_time`, status must be one of a fixed list.
- **ON DELETE CASCADE** removes a user's addresses, items and orders with them; **ON DELETE SET NULL** keeps an order's history when an item is deleted.
- **Prices are copied into `order_items`** (`unit_price`), so changing a price later doesn't change old orders.

Small additions to the ERD: `users.password`, `verification_documents.file_url`, and `order_items.slot_id` (which slot a service order books).

5. Features, one by one

Feature	Endpoints	How it works
Register & login	`POST /api/auth/register`, `/login`, `GET /auth/me`	Register creates a `users` row plus a `customers` or `service_providers` row in one transaction. Login checks the bcrypt hash and returns a JWT. Admins need `admin_secret`.
Provider verification	`POST /api/documents`, `PATCH /api/providers/:id/verification`	New providers are `pending`. They upload document links; an admin accepts or rejects. Only `accepted` providers can create items (`verifiedProvider` middleware).
Addresses	`/api/addresses`	Full create/read/update/delete, only on your own addresses.
Categories	`/api/categories`	Anyone reads; only admins create, edit, delete. Three are seeded on start.
Food, crafts, services	`/api/foods`, `/api/crafts`, `/api/services`	One shared router (`routes/items.js`) serves all three. Search with `q`, `category_id`, `provider_id`, `min_price`, `max_price`, `sort`.
Appointment slots	`/api/slots`	Providers add time slots to their services; booking one sets `is_booked`.
Orders	`POST /api/orders`, `PATCH /api/orders/:id/status`	See below.
Payments	`/api/payments`	Customer records cash or card for the order total; the provider marks it `success` or `failed`. No real card gateway.
Reviews	`/api/reviews`	Only for your own **completed** order, once. After every add/edit/delete the provider's average `rating` is recalculated.

Feature	Endpoints	How it works
Chat	`/api/conversations`, `/:id/messages`	A customer opens a chat with a provider; reading messages marks them read; the list shows `unread_count`.

Placing an order (the most important flow). `Order.create` in `src/models/Order.js` runs inside one database **transaction**:

1. Check every item exists, belongs to an accepted provider and is available.
2. Check all items come from the same provider.
3. For food/craft: check and reduce `stock_quantity`. For services: book the slot (fails if already booked).
4. Compute prices: line = unit price × quantity; service unit price = hourly rate × hours needed; add a 20 delivery fee if there is food or craft.
5. Insert the `orders` row and its `order_items`.

If any step fails, everything is rolled back: no half-orders, no stock lost.

Order status rules: provider moves `pending → accepted → in_progress → completed` (or cancels while pending); the customer can cancel only while `pending`. Cancelling puts the stock back and frees the slot.

6. Security and validation

Security works in layers; each one catches something different.

- **Passwords** are hashed with bcrypt (10 rounds) and never sent back: the `User` model only selects safe columns.
- **JWT:** after login the app sends `Authorization: Bearer <token>`. The `auth` middleware verifies it and puts `user_id`, `user_type`, `customer_id` / `provider_id` on `req.user`. Tokens last 7 days.
- **Roles:** `role('admin')`, `role('provider')` … block the wrong account type with **403**.
- **Ownership:** routes load the row first and compare its owner with the logged-in user; someone else's address, item or review answers **404** (we don't even confirm it exists).
- **Mass assignment protection:** each model lists which columns a request may set (`crud.js` *editable* vs *fixed*). Sending `provider_id: 999` in a body is ignored.
- **SQL injection:** every value goes in as a `$1` parameter, never pasted into the SQL string.
- **Database constraints** are the last line: CHECK, NOT NULL, UNIQUE, FOREIGN KEY.
- **Public data stays safe:** `national_id` and passwords never appear in public endpoints.

One error handler in `src/app.js` turns problems into clear HTTP codes:

Code	Meaning	Example
400	Bad input	Negative price, wrong status change, `abc` as an id
401	Not logged in	No or expired token
403	Logged in, not allowed	Customer creating a category
404	Not found or not yours	Editing another provider's item
409	Duplicate	Email already registered, second review of an order
500	Our bug	Logged on the server, generic message to the client

7. Testing and demo data

The backend is checked three ways.

1. **Automated test** — `npm test` runs `test/api.test.js`. It starts a throwaway Postgres (PGlite), then walks the whole story: register → verify → sell → order → pay → review → chat, plus the failure cases (wrong password, out of stock, double-booked slot, wrong role). It never touches the real database.
2. **Postman collection** — `postman_collection.json`, 64 requests in 11 folders, each with a status check. Run it with the Collection Runner; it saves tokens and ids between requests automatically. The last folder, *Cleanup*, deletes its test data.
3. **Demo data** — `npm run seed` wipes the database and loads realistic sample data from `src/seed.sql`.

Demo accounts (password for all: `password123`):

Email	Role	What's there
admin@example.com	Admin	Verifies providers, manages categories
mona@example.com	Provider (accepted)	Mona's Kitchen: 4 dishes, rating 5.0
hoda@example.com	Provider (accepted)	Hoda Handmade: 3 craft items
salma@example.com	Provider (accepted)	Salma Home Services: 2 services, 5 slots, rating 4.0

Email	Role	What's there
nour@example.com	Provider (pending)	Waiting for admin approval
sara@example.com	Customer	Orders in several statuses, a review, a chat
omar@, laila@, karim@example.com	Customers	Orders, payments and chats

8. Running the demo

Setup, once per machine:

1. Install Node.js and PostgreSQL; create a database named `women_platform`.
2. In the project folder run `npm install`.
3. Copy `.env.example` to `.env` and fill in `DATABASE_URL` (your Postgres password), `JWT_SECRET` and `ADMIN_SECRET`.

On evaluation day (about 10 minutes):

1. `npm run seed` — load the demo data.
2. `npm run dev` — start the server; open `http://localhost:3000/api/health` and show `{"status":"ok"}`.
3. `npm test` — show the automated test passing.
4. In Postman, run the collection (skip *Cleanup*) and show 64 green requests.
5. Walk through one story by hand: log in as nour@example.com, try to create a food item (403, not verified) → log in as admin, accept her → she creates the item → log in as sara, order it → Nour accepts and completes it → Sara reviews it.
6. Open pgAdmin → `women_platform` → Tables and show the new rows in `orders`, `order_items` and `reviews`.
7. Open `src/schema.sql` and one model (e.g. `src/models/Order.js`) and explain the code.

9. Questions the evaluator may ask

What is a REST API? A set of URLs the app calls with HTTP methods: GET reads, POST creates, PATCH updates, DELETE removes. Data goes back and forth as JSON.

What is a model? A route? Middleware? A model talks to the database (one per table, in `src/models`). A route maps a URL to code and sends the response (`src/routes`). Middleware runs before the route, e.g. checking the login token.

Why don't you have controllers and services? Our routes play the controller role, and the business rules live in the models (e.g. `Order.create`). For this size, extra layers would only pass calls through. They are easy to add if needed.

What is CRUD and where is it? Create, Read, Update, Delete. `src/models/crud.js` gives every simple table these four operations, so each model file is only a few lines.

What is a transaction and where do you use it? A group of SQL statements that all succeed or all fail. We use it for registration, placing an order, cancelling an order and sending a chat message.

How do you stop two customers buying the last cake at the same time? Inside the order transaction we lock the item row (`SELECT … FOR UPDATE`), check stock, then reduce it. The second order waits, then sees 0 left and fails.

Why hash passwords instead of encrypting them? Hashing is one-way. Even we can't read a password; at login we hash what was typed and compare.

What does the JWT contain? Can a user change it? Their ids and role, signed with `JWT_SECRET`. Any change breaks the signature, so the server rejects it.

How does the provider rating update? After each review is added, edited or deleted, one SQL statement sets `rating` to the rounded average of that provider's reviews.

What happens when an item that was ordered gets deleted? The order line keeps its price and quantity; the item id becomes NULL (`ON DELETE SET NULL`), so history survives.

What is missing / future work? Real card payments (e.g. a payment gateway), file upload for documents and images, push notifications, loyalty point rules, and the mobile app itself.